

Föreläsning 20

Ändrad
9 Oct
22:29

struct

Ofta hör variabler ihop: de hjälps åt att beskriva samma sak. När detta är fallet vill vi också kunna behandla dessa variabler i ett enda "paket".

Vi vill ha variablerna liggandes i en "låda", så att vi enkelt kan flytta omkring dem tillsammans.

I nästa exempel definierar jag en låda som håller reda på allt som är värt att veta om en student. Lådan heter `student`.

```

1  #include <stdio.h>
2
3  typedef struct {
4      char name[40];           /* Ex: "Tångring, Jan" */
5      char user[8];           /* Ex: "di97jtn" */
6      char program[6];        /* Ex: "TIDAY" */
7      unsigned long birthdate; /* 97-07-02 == 19970702 */
8      short control;          /* Sista fyra siffrorna */
9  } student;
10
11
12 void print_pnr(student s)
13 {
14     int day, month, year;
15     day = s.birthdate % 100;
16     month = (s.birthdate / 100) % 100;
17     year = (s.birthdate / 10000) % 100;
18     printf("%.2d%.2d%.2d-%.4d\n", year, month, day, s.control);
19 }
20
21
22 void print_student(student s)
23 {
24     printf("Namn: %s (%s)\n", s.name, s.user);
25     printf("Personnummer: ");
26     print_pnr(s);
27     printf("Program: %s\n\n", s.program);
28 }
29
30
31 void main()
32 {
33     student s1 = {"Tångring, Jan", "di97jtn",
34                  "TIDAY", 19630806, 6635};
35     student s2 = {"Olsson, Urban", "ei96uon",
36                  "TIELY", 19830917, 5550};
37
38     print_student(s1);
39     print_student(s2);
40 }

```

En körning ger:

```

1  Namn: Tångring, Jan (di97jtn)
2  Personnummer: 630806-6635
3  Program: TIDAY
4
5  Namn: Olsson, Urban (ei96uon)
6  Personnummer: 830917-5550
7  Program: TIELY
8

```

■ Först definierar vi lådans utseende. Vi talar om vilka

värden som ska ligga i lådans olika "fack". Det gör vi i en *typedefinition* på rad 3--9.

Det heter dock inte "låda", utan `struct`. Och vår `struct` heter `student`. Lådan har fem förvaringsfack: tre strängar av olika storlekar, en `unsigned int`, samt en `short int`.

Dessa variabler lagras konsekutivt i minnet, precis som variablerna i ett fält. Sammanlagt tar vår `struct` upp $40+8+6+4+2 = 60$ byte.

Notera syntaxen i typedefinitionen:

1. Nyckelordet `typedef`
2. Nyckelordet `struct`
3. Inramningen med `{` och `}`
4. Mellan parenteserna: ett antal vanliga variabeldeklARATIONER.
5. Till sist `struct`ens namn -- `student`.

En vanlig hederlig typ

Ett praktiskt och vackert faktum är att vår egendefinierade typ har samma status och "rättigheter" som de inbyggda typerna `float`, `int`, etc. Vi kan till exempel:

- deklarerar variabler av typen `student`

```
student s
```

- ha värden av typen `student` som argument till funktioner

```
void print_pnr(student s)
```

- eller låta funktioner returnera en `student`

```
student fgetstudent(FILE *f);
```

- deklarerar `student`-pekare

```
student s1, s2, *ptr = &s1;
```

- inget hindrar oss från att använda tilldelning på vanligt sätt (såvida vi inte använder en föråldrad C-kompilator):

```
s2 = s1; /* Hela s1 kopieras till s2! */
```

- vi kan till och med ha `student` som element i ett fält

```
student s[80], *ptr = &s[3];
```

(80 stycken `student`-variabler gör $80 \cdot 404$ byte = 32320 byte)

En helt vanlig hederlig *typ* med andra ord -- `student`.

Punktnotation

För att komma åt "facken" i vår `struct` använder vi en `punkt`:

```
student s1, s2, s[80];
s1.name = "Ollson, Fridolf";
s1.name[2] = 's';          /* "Ollson" --> "Olsson" */
s1.user = "di97fon@cs.umu.se";
s1.birthdate = 25 + 100 * (11 + 100 * 1932); /* 25 nov 1932 */
s1.control = 5445;
```

Fält + storlek = sant!

Ett vanligt användningsområde för en `struct` är att hantera ett fält och dess aktuella storlek som ett enda värde. Jag ska justera första exemplet i [föreläsning 18](#) så att detta blir fallet.

```
1  #include <stdio.h>
2
3  #define MAXARRAY 100
4
5  typedef struct {
6      unsigned length;
7      float element[MAXARRAY];
8  } float_array;
9
10
11 void copy(float_array *x, float_array y)
12 {
13     int i;
14     for (i = 0; i < (*x).length; i++)
15         (*x).element[i] = y.element[i];
16 }
17
18
19 void print(char *text, float_array x)
20 {
21     int i;
22     printf("%s", text);
23     printf("(");
24     for (i = 0; i < x.length-1; i++)
25         printf("%g, ", x.element[i]);
26     if (x.length > 0)
27         printf("%g", x.element[i]);
28     printf(")\n");
29 }
30
31 void main()
32 {
33     float_array x = {5, {1, 2, 3, 4, 5}};
34     float_array y = {5, {2.2, 2.2, 2.2, 2.2, 2.2}};
35
36     print("x = ", x);
37     print("y = ", y);
38
39     copy(&y, x);
40
41     print("x = ", x);
42     print("y = ", y);
43 }
```

Programmet fungerar:

```
1  x = {1, 2, 3, 4, 5}
2  y = {2.2, 2.2, 2.2, 2.2, 2.2}
3  x = {1, 2, 3, 4, 5}
4  y = {1, 2, 3, 4, 5}
```

- Ett av argumenten till `copy` är en `student`-pekare. Annars skulle ingenting bli kopierat. Notera hur vi kommer

åt fälten inuti funktionen, exempelvis:

```
(*x).length
```

Men det hade du själv kunnat räkna ut! Du vet redan att

- "*" används för att komma åt variabler via pekare, och att

- "." används för att komma åt fälten i en `struct`.

- Eftersom man ofta vill komma åt fälten i en `struct` via pekare, finns en mer praktisk kortform:

```
x -> length
```

- Egentligen behövs inte min funktion `copy()`, eftersom tilldelning fungerar mellan `structar`:

```
y = x;
```

Å andra sidan är `copy` smartare än vanlig tilldelning, eftersom den bara kopierar `x.length` element. Och vanlig tilldelning kopierar samtliga fält.

Att `x.length` anger aktuell längd är min -- programmerarens -- egen konvention. Det kan inte datorn veta något om.

Samma lektion

